1. Title Page

Lab Name: Lab 14: Transport Layer Security (TLS) **Date:** YYYY-MM-DD **Name:** [Your Name] **Lab Partners:** [Partners' Names, if any] **Software/Hardware Used:** Cisco Modeling Labs (CML) - Personal / Free Edition

---

2. Background

Transport Layer Security (TLS), and its predecessor Secure Sockets Layer (SSL), are cryptographic protocols designed to provide security over a computer network. TLS operates on top of transport layer protocols (like TCP) and below application layer protocols (like HTTP), forming **HTTPS**.

TLS provides three core security services:

1. **Confidentiality**: All data transferred between the client and server is encrypted using a negotiated symmetric key algorithm (e.g. AES).
2. **Authentication**: The server (and optionally the client) proves its identity using digital certificates signed by a trusted Certification Authority (CA).
3. **Data Integrity**: Packets are cryptographically signed using a Message Authentication Code (MAC) to prevent tampering.

The **TLS Handshake** is where these services are negotiated. In a classic TLS 1.2 handshake:

- **Client Hello**: Sent by the client to propose TLS version, random bytes, and supported cipher suites.
- **Server Hello**: Sent by the server to select the TLS version, server random bytes, and the chosen cipher suite.
- **Certificate Exchange**: The server sends its public key certificate.
- **Server Key Exchange / Server Hello Done**: Sends parameters and signals the end of the hello phase.
- **Client Key Exchange**: The client sends key agreement parameters (e.g. Pre-Master Secret encrypted with the server's public key).
- **Change Cipher Spec**: Both parties signal that future traffic will be encrypted.
- **Encrypted Handshake (Finished)**: Verifies that the handshake was not tampered with.
- **Application Data**: Encrypted HTTP messages.

In this lab, you will configure a local Nginx web server with SSL support inside CML, generate real HTTPS traffic from an Alpine client, and analyze the resulting TLS handshake and record structures in Wireshark.

---

3. Objective / Purpose

To deploy a secure web server (HTTPS) in CML, capture the negotiation of the TLS handshake, analyze TLS record structures and parameters in Wireshark, and understand how public-key cryptography and digital certificates establish secure channels.

---

4. Topology Diagram

```
graph LR
    H0["Host H0 (Client)<br>IP: 192.168.1.10/24<br>eth0"] <--> |"L2 Link"| H1["Host H1 (Nginx Web
Server)<br>IP: 192.168.1.20/24<br>eth0"]

    classDef node fill:#d4f1f9,stroke:#007b8b,stroke-width:2px;
```

```
    classDef server fill:#fff2cc,stroke:#d6b656,stroke-width:2px;

    class H0 node;
    class H1 server;
```

Details:

- Client host (`<initials>-H0`) connected directly to Nginx Web Server (`<initials>-H1`).
- Both interfaces are configured in the `192.168.1.0/24` subnet.

---

5. Equipment & Environment

- **Software:** Cisco Modeling Labs (CML), Terminal Emulator, Wireshark, OpenSSL, Nginx
- **Hardware / Virtual Nodes:** 2x Linux Nodes (configured as Host H0 and Nginx Web Server H1)

---

6. Configuration / Methodology

IP Addressing Table

Device	CML Node Name	Interface	IP Address	Subnet Mask	Services Enabled
Client	`<initials>-H0`	eth0	192.168.1.10	255.255.255.0	Curl (Client)
Web Server	`<initials>-H1`	eth0	192.168.1.20	255.255.255.0	Nginx (HTTPS: 443)

Step-by-Step Configuration Guide

1. Configure Host H0 (`<initials>-H0`)

Click on H0, open the **Edit Config** tab, and enter the IP configuration:

```
ip addr add 192.168.1.10/24 dev eth0
```

2. Configure Nginx Web Server (`<initials>-H1`)

Click on H1, open the **Edit Config** tab, and enter the base configuration:

```
ip addr add 192.168.1.20/24 dev eth0
```

To enable HTTPS, we must configure Nginx with an SSL certificate. Open H1's **Console** tab and execute:

1. **Generate a Self-Signed Certificate and Key**:

   ```
   openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
     -keyout /etc/ssl/private/nginx-selfsigned.key \
     -out /etc/ssl/certs/nginx-selfsigned.crt \
     -subj "/CN=CS457-Server/O=CS457/C=US"
   ```

2. **Configure Nginx for SSL**: Create or edit `/etc/nginx/conf.d/default.conf` to listen on port 443 with SSL enabled:

```
server {
    listen 443 ssl;
    server_name localhost;

    ssl_certificate /etc/ssl/certs/nginx-selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/nginx-selfsigned.key;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
```

3. **Start/Restart Nginx**:

```
nginx -s reload || nginx
```

Step-by-Step Traffic Generation & Execution:

1. **Start Packet Capture**: Right-click the link connecting **Host H0 and Server H1**, select **Packet Capture**, and click **Start**.
2. **Generate HTTPS Request**: Open H0's console and send an HTTPS request using `curl` (the `-k` flag allows connecting despite the self-signed certificate):

   ```
   curl -k -v https://192.168.1.20/
   ```

   (Ensure you see a successful HTML response in the output).
3. **Download Capture**: Stop the CML packet capture and download the `.pcap` capture file to your local computer.

---

7. Wireshark Analysis and Questions

Open your downloaded packet capture in Wireshark and answer the following questions:

Part A: TCP Handshake vs. TLS Setup

1. What is the packet number in your trace that contains the initial **TCP SYN** message?
2. Is the TCP connection fully established (3-way handshake) before or after the first TLS message is sent from the client to the server? Refer to packet numbers to justify your answer.

Part B: The TLS Handshake - Client Hello

3. What is the packet number in your trace that contains the TLS **Client Hello** message?
4. What version of TLS is your client running, as declared in the Client Hello message?
5. How many **Cipher Suites** are supported by your client, as declared in the Client Hello?
6. Your client generates and sends a string of "random bytes" to the server in the Client Hello message.
   - What are the first two hexadecimal digits in the `random bytes` field (excluding the GMT Unix Time subfield if present)?
7. What is the purpose of this "random bytes" field? (Refer to Slide 4 or RFC 5246 section 8.1).

Part C: The TLS Handshake - Server Hello & Certificates

8. What is the packet number in your trace that contains the TLS **Server Hello** message?
9. Which cipher suite was chosen by the server from the options offered in the Client Hello?
10. Does the Server Hello message contain random bytes, similar to the Client Hello? If so, what is their purpose?
11. What is the packet number of the TLS record containing the server's **Certificate**?
12. Inspect the certificate returned by the server:
    - What is the Common Name (CN) or identity bound to this certificate? (Check the `id-at-commonName` field).
13. What is the name of the Certification Authority (CA) that issued the certificate? (Since this is a self-signed setup, explain why the Issuer matches the Subject).
14. What digital signature algorithm was used to sign this certificate? (Check the `signature` subfield of the certificate).
15. What are the first four hexadecimal digits of the public key's **modulus**?
16. Do you see any packets exchanged between your client and a public Certificate Authority (CA) in the capture? If not, explain why the client did not contact a CA.
17. What is the packet number in your trace containing the **Server Hello Done** record?

Part D: Handshake Completion & Application Data

18. What is the packet number containing the **Client Key Exchange**, **Change Cipher Spec**, and **Encrypted Handshake Message** being sent from the client to the server?
19. Does the client provide its own CA-signed public key certificate back to the server?
20. What symmetric key cryptography algorithm is being used by the client and server to encrypt the application data?
21. In which of the TLS messages is this symmetric key algorithm finally decided and declared?
22. What is the packet number in your trace of the first encrypted message carrying **Application Data**?
23. What is the assumed plaintext content of this first encrypted application data packet? (Given that this trace was generated by fetching the homepage of the server).
24. What packet number contains the client-to-server TLS record shutting down the connection? (Look for an encrypted alert packet, e.g. `close_notify`).

---

8. Troubleshooting

Issue Encountered: [Describe what went wrong, if anything] **Discovery Method:** [How did you find the issue?] **Resolution:** [How did you fix it?]

(If no issues were encountered, explain potential pitfalls for this configuration).

---

9. Conclusion & Analysis

Summarize your findings. Explain the structural sequence of the TLS handshake, how symmetric and asymmetric encryption are combined to establish a secure channel, and the role of digital certificates in preventing man-in-the-middle attacks.

---

10. What to Hand In

- The Lab YAML configuration file with your name, date, and name of the lab at the top in comments.
- A single PDF report containing:

- Your written answers to the 24 Wireshark analysis questions in Section 7.
- A screenshot of your active CML topology showing the running hosts (`<initials>-H0` , `<initials>-H1`).
- The required screenshot of your server's Nginx configuration file displaying the SSL certificate paths.
- An annotated screenshot of your Wireshark capture window showing the Client Hello and Server Hello TLS handshake records.